Omnichannel Payment Processing System

Document Type: Initial Client Discovery & Functional Requirements Specification
Project Context: Scrum Foods On-Demand Delivery Ecosystem
Prepared By: Business Analysis Team
Audience: System Architects, Technical Leads, Development Team, and Stakeholders

1. Background & Discovery Context

During the initial requirements elicitation session with client stakeholders for the platform checkout and billing module, a primary operational objective was identified: providing customers with a seamless, flexible, and secure omnichannel checkout experience. The client emphasized that transaction drop-offs frequently occur when preferred payment modes are unavailable or when payment processing is cumbersome.

To solve this, the core payment processing engine must support four distinct payment instruments: **Credit/Debit Card**, **Digital Wallet**, **Cash on Delivery (COD)**, and **Net Banking**. The system must orchestrate verification, third-party gateway redirection, authorization, and immediate order settlement while adhering to enterprise 3-tier architectural standards.

2. Case Study Problem Statement

A customer finalized an order and proceeded to the platform checkout stage. The platform must present available payment options, validate the selected payment method, coordinate with internal account ledgers or external financial gateways, and return a real-time transaction confirmation or failure notice.

Specifically:

- A customer can make a payment either by **Card**, **Wallet**, **Cash**, or **Net Banking**.
- If **Card** is selected, the system must collect and validate card credentials via a secure payment gateway integration.
- If **Wallet** is selected, the platform checks internal or third-party wallet balances and deducts the amount upon customer authorization.
- If **Cash** is chosen (Cash on Delivery), the transaction state must register as pending collection, assigning a collection responsibility to the delivery executive upon fulfillment.
- If **Net Banking** is selected, the system must allow the customer to select their preferred banking institution from an approved list, redirect the session to the secure third-party bank payment gateway, receive asynchronous status callback upon completion, and update order/payment records accordingly.

3. Detailed Functional Requirements

Requirement ID	Requirement Title	Functional Description
FR-PAY-01	Payment Method Selection	The checkout screen shall present single-selection options: Card, Digital Wallet, Cash on Delivery, and Net Banking.
FR-PAY-02	Net Banking Financial Institution Elicitation	Upon choosing Net Banking, the interface shall populate a list of participating banks for customer selection.
FR-PAY-03	Gateway Routing & Session Handshake	The system shall securely redirect the customer to the selected bank's authentication gateway, passing encrypted transaction metadata (Order ID, Amount, Timestamp).
FR-PAY-04	Asynchronous Callback Processing	The system shall listen for secure callback responses from the bank system (Success, Failure, Timeout) and verify payload integrity.
FR-PAY-05	State Persistence & Ledger Updates	The system shall atomically record transaction outcomes in the payments entity and update the order state from 'Initiated' to 'Paid' or 'Failed'.
FR-PAY-06	Notification Dispatch	The platform shall generate receipt notifications via SMS/email and present a transaction confirmation view to the user.

4. Non-Functional & Architectural Constraints

- **Security Compliance:** All payment communications must execute over TLS 1.3 with SHA-256 payload encryption. Sensitive banking credentials must never be stored on internal application servers (PCI-DSS & RBI compliance guidelines).
- **Architectural Separation:** Clean separation of concerns adhering to standard 3-tier enterprise patterns (Presentation, Application/Business Logic, Data Layer).
- **Reliability & Idempotency:** Payment requests must support idempotency keys to prevent duplicate deductions during network latency or multiple submission attempts.

5. Analysis & Modeling Requirements (Deliverables Checklist)

Based on the elicited case study specifications, the Business Analyst must formulate, structure, and deliver the following architectural and analytical deliverables:

1. **Use Case Diagram:** Model the system boundary, primary actor (Customer), secondary actors (Payment Gateway, Core Banking System), and use cases covering the four payment mechanisms with appropriate relationships (*«include»*, *«extend»*, or generalization).
2. **Class Stereotype Classification:** Derive and tabulate the complete set of **Boundary Classes** (UI/Screen interfaces, Gateway Handlers), **Controller Classes** (Business Logic, Transaction Routers), and **Entity Classes** (Persistent database domain models).
3. **3-Tier Architectural Mapping:** Map each identified Boundary, Controller, and Entity class into its corresponding architectural layer (Presentation Tier, Application Tier, and Data Tier).
4. **Domain Model Formulation:** Define and elaborate the Domain Model specifically for the **Customer making payment through Net Banking** scenario, documenting domain entities, attributes, and relationships.
5. **Sequence Diagram Specification:** Develop a complete UML Sequence Diagram illustrating the synchronous and asynchronous chronological message exchanges between Actor, Boundary, Controller, External Bank Gateway, and Database Entity during a Net Banking transaction.
6. **Conceptual Model Definition:** Explain the high-level Conceptual Model representing static business concepts, cardinality, and business rules governing omnichannel payment handling.